

Design of a Flight Ticket Price Data Pipeline and Storage System Using Apache Spark

Ain Nurnabila
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
A23CS0207

Abstract—Airline ticket prices change constantly with demand, booking time, route and season, making it hard for both travelers and travel platforms to acknowledge the pricing patterns using spreadsheets or the old-style manual analysis alone. This proposal presents the design of an individual data engineering project which builds an end-to-end data pipeline and storage system for flight ticket price analysis by using Apache Spark. The pipeline integrates three publicly available data sources which are a historical flight booking price dataset, live weather data obtained through an application programming interface (API), and a flight delay and airport-performance dataset into a cleaned and analysis-ready dataset. Spark is used for distributed data ingestion, cleaning, transformation and feature engineering, after which the processed data is stored in one structured format that suitable for analytical querying and is enhanced with a machine learning algorithm to predict ticket prices better. This report documents the design, implementation, and evaluation of this pipeline, including the trained price-prediction model and resulting analytics dashboard.

Index Terms—data pipeline, Apache Spark, flight price prediction, ETL, data Lakehouse, machine learning

I. INTRODUCTION

This section introduces the domain of the project, states the core problem being addressed, and outlines the objectives the project sets out to achieve.

A. Background and Problem Identification

The domain selected for this project is the airline's retail and travel-technology domain, particularly the pricing of domestic flight tickets. Online travel platforms such as Ease My Trip publish fares that change several times a day regarding the airline, the time of departure, the route, the number of stops, and how many days remain before departure. For a normal and ordinary traveler, this volatility makes it hard to decide whether a fare shown today is reasonable or whether waiting a few more days would be affordable. For the platform operators and airlines themselves, the same volatility, together with the operational disruptions like adverse weather and flight delays, makes it way more difficult to consolidate performance, pricing, and environmental data into one place for analysis.

A clear and specific problem can therefore be stated as follows: existing flight booking platforms expose fare, schedule and delay information separately, in inconsistent formats and update frequencies, with no unified, automated pipeline that cleans, integrates, and stores this data for downstream

price analysis and prediction. Building this specific pipeline is very relevant as the Malaysian and regional travel industry increasingly relies on data-driven dynamic pricing, and as a working understanding of large-scale, distributed data engineering using Apache Spark is straightly transferable to the kind of Bronze-Silver-Gold-Curated, medallion-style pipelines used in industry.

B. Project Objectives

The main goal of this project is to design and build a Spark-based ETL pipeline that ingests flight price, flight delay and performance, and weather data from three different source types into a unified schema. Then, clean, transform, and store the integrated dataset in a structured analytical storage layer suitable for querying and visualization. Next, to enhance the pipeline with a machine learning regression algorithm that predicts flight ticket prices and evaluates its accuracy using standard regression metrics.

II. LITERATURE REVIEW/RELATED WORK

The remaining question is why delay and route data need to be in the pipeline at all, rather than just using the pricing data on its own. Two studies address this from quite different angles. Bubalo and Gaggero [2] looked at how delays spread through European airline networks and found something interesting like on route where low-cost carriers increased competition, ticket prices and delay levels fell together. This suggests that operational reliability and pricing are both not independent as they move together regarding the competitive environment on a given route. Calzada and Fageda [3] came to the same question from a different direction, looking at how dominant airports and concentrated route markets affect scheduling flexibility. Their finding was that when one carrier controls most of the traffic at an airport, scheduling turns out to be less efficient, delays increase, and the resulting higher operating costs eventually show up in fares. So while Bubalo and Gaggero [2] focused on what happens when competition increases, Calzada and Fageda [3] focused on what happens when it does not, and both ended up pointing to the same summarization that delay patterns and ticket prices are closely tied. For this project, that means using price data alone would give an incomplete picture. Including flight delay records as a second data source allows the pipeline to capture those

operational signals alongside pricing data, which makes the eventual price prediction model more grounded in the actual factors that drive fare changes.

However, one thing none of these studies do is bring weather data into the pipeline as a direct variable. This is a gap worth addressing, since adverse weather is one of the most common triggers for operational disruptions that feed into pricing decisions. This project fills that gap by querying the Open-Meteo Historical Weather API as a third data source, attaching delay temperature, precipitation and wind speed at each departure city to the booking records, so that weather context is available as an input feature in the prediction model alongside delay and pricing data.

Wong et al. [6] provided further support for the modelling approach adopted here, applying Spark-based machine learning models to a large-scale flight pricing dataset and finding that ensemble tree-based regressors consistently outperformed simple linear baselines in predictive accuracy. This finding directly informed the decision in Section III-F to train a tree-based ensemble regressor rather than relying on linear regression alone, and the results obtained in this project are consistent with that precedent.

III. METHODOLOGY

This section explains the project methodology, the data sources used, and the steps taken to ingest, process, store, and model the data.

A. Selected Methodology

Flight ticket prices are not determined by a single factor. From a business perspective, fares shift because of several interacting variables like the route between source and destination city, the travel class, the time of departure, the number of stops, the airline’s competitive position on that route, the number of days remaining before departure, and the current weather conditions at the source city during the travel period. For example, routes served by only one or two carriers usually carry higher average fares due to reduced competition, while last-minute bookings consistently attract a price premium as available seats become insufficient. Similarly, adverse weather at departure cities has been shown in the literature to increase operational disruptions, which indirectly drives costs and influences dynamic pricing decisions. The business challenge this project stated is that these factors are spread across separate, inconsistent data sources: booking platforms hold pricing and schedule data, meteorological services hold weather data, and aviation authorities hold delay records, making it impossible for a single analyst to consolidate and analyze them without an automated pipeline that ingests, cleans, and integrates all three into a unified schema.

To address this challenge, this project adopts the Agile methodology, implemented as sequential sprints rather than the overall Waterfall approach. Agile is chosen over Waterfall for two specific reasons: the project deliverable is a working Apache Spark pipeline built from three heterogeneous sources, and issues such as mismatched schemas, missing fields, or

API rate limits are realistically discovered only once ingestion code is run, so a Waterfall plan that fixes all requirements upfront would be difficult to follow. Agile’s iterative cycle allows each pipeline stage to be developed, demonstrated, and refined as a separate sprint, with working software produced at the end of every sprint rather than only at the very end of the project, consistent with how Spark-based pipeline development is described by Mandala [4] and Wong et al. [6].

The project is organized into four sprints, summarized in Table I, each ending with a runnable increment of the pipeline that is tested before the next sprint begins.

TABLE I: Project Sprint Breakdown

Sprint	Focus	Increment Delivered
1	Source setup & ingestion	All three sources loaded into Spark Bronze layer
2	Cleaning & transformation	Joined, deduplicated Silver layer dataset
3	Storage & aggregation	Gold and Curated layers with analytical tables
4	AI model integration	Trained Spark MLlib price-prediction model with evaluation results

B. Data Source Identification

Three different, publicly accessible data sources were selected so that the pipeline must reconcile different formats and update frequencies, as required by the project brief.

Firstly is a CSV file of Flight Price Prediction dataset, which is a primary dataset. Published on Kaggle by Bathwal [1], this dataset contains 300,153 cleaned domestic flight booking records scraped from the Ease My Trip platform between 11 February and 31 March 2022, covering six Indian metro cities. It provides the core pricing features used in this project which are airline, source/destination city, departure and arrival time bins, number of stops, travel class, duration, days left before departure, and price.

Next, an open dataset of Flight Delay Dataset 2018–2022. Published on Kaggle by Mulla [5], this dataset consists of domestic flight records from the U.S. Bureau of Transportation Statistics covering the year 2022. It contains flight delay records involving departure delay, arrival delay, weather delay minutes, and cancellation status per flight, which is used to enrich the pipeline with operational reliability features that the literature in Section II links to pricing behavior.

Lastly is an API data from Open-Meteo Historical Weather API. Open-Meteo is a free, open-source weather API [7] (DOI: 10.5281/zenodo.7970649) which returns hourly temperature, precipitation and wind-speed data for any coordinate and date range without requiring an API key. It was queried programmatically from within the Spark pipeline to attach simultaneous weather conditions to the source-city and date of each flight record, supplying the live, API-based data source required by the project brief.

C. Data Ingestion

The ingestion plan maps all three sources onto a common schema keyed by city and date. The Kaggle CSV is read directly into a Spark DataFrame. The Kaggle delay dataset is downloaded once, cached locally and read in the same way, and the Open-Meteo API is called through Spark orchestrated HTTP requests, batched by unique city-date pairs to avoid redundant calls, with the JSON response parsed into a DataFrame. Missing values are handled per source: numeric gaps are imputed using the median of the same city and month, while categorical gaps are imputed with an explicit “Unknown” category rather than being dropped, so that no booking record is discarded solely due to a missing operational field. Records that cannot be matched to a valid city-date pair across all three sources are logged separately rather than silently removed, so that the size of any data loss remains traceable.

D. Data Processing/Transformation

Within Spark, the cleaning stage removes exact duplicate booking rows, standardizes categorical text to a single casing and spelling convention, and converts the departure/arrival time bins and days-left field into numeric and ordinal types suitable for modelling. Outlier prices are flagged rather than deleted, so that the AI model in Section III-F can be tested with and without them. The cleaned, joined dataset is then transformed into a structured, denormalized analytical table with one row per flight booking and aligned weather and delay-context columns, ready for loading.

E. Data Storage

The processed data was stored using a four-layer medallion pattern. Raw ingested files are kept unmodified in a Bronze layer for traceability. The cleaned and joined output of Section III-D is written in a compressed columnar format (Parquet) as a Silver layer. A Gold layer then holds business-aggregated tables. Finally, a Curated layer republishes selected Gold tables in a stable, documented, query-optimized form intended for direct consumption by the price-prediction model and any downstream dashboard, decoupling internal pipeline changes from what external consumers query. A medallion pattern is preferred here over a single flat warehouse table because the project must retain the original semi-structured API responses for audit purposes while still supporting fast, stable analytical SQL queries, which Spark and Parquet support natively across all four layers.

F. Artificial Intelligence Algorithm

To enhance the pipeline beyond plain ETL, a supervised regression algorithm was integrated at the end of the Spark pipeline to predict flight ticket prices from the integrated features. Following the precedent in [2], a tree-based ensemble regressor was trained and compared against a simple linear regression baseline, with the data split chronologically into training and testing sets so that the model is evaluated on dates it has not seen, and accuracy reported using RMSE and R^2 .

G. Risk Analysis

A few risks were identified during the pipeline planning, together with the mitigation strategy adopted for each.

- **Data quality risk:** the delay dataset originates from US domestic flights rather than Indian routes, which means delay figures are industry-level proxies rather than route-specific measurements. This was mitigated by treating delay features as general operational-reliability signals rather than exact route-level figures, and by documenting this limitation explicitly rather than presenting the joined data as ground truth.
- **API reliability risk:** the Open-Meteo API may return incomplete responses or be rate-limited during batch calls. This was mitigated by batching requests on unique city-date pairs to minimize call volume, and by logging any unmatched city-date records separately instead of allowing a failed call to silently drop booking rows.
- **Schema and integration risk:** the three sources use different field names, granularity, and casing conventions, which could cause silent join mismatches. This was mitigated by standardizing categorical text and key formats during the Silver-layer cleaning stage before any join was performed.
- **Tooling and environment risk:** running large joins and model training on Google Colab risks session timeouts or memory limits given the size of the delay dataset. This was mitigated by checkpointing intermediate output to Parquet at each medallion layer, so a session interruption does not require re-running the entire pipeline from raw ingestion.

IV. SYSTEM DESIGN AND SYSTEM ARCHITECTURE

This section presents the system’s design through its use cases, architecture, database structure, and main interface.

A. System Overview and Use Case

The primary actor of the system is the project administrator, who triggers pipeline runs and reviews outputs. Meanwhile, a secondary actor is the data analyst who queries the curated storage layer for price insights. The main use cases are triggering data ingestion from the three sources, running data cleaning and transformation, load curated data into storage, training and evaluating the price-prediction model, and viewing summary analytics.

B. System Architecture

Fig. 2 summarizes the planned end-to-end flow, which involves the three data sources being ingested into a raw Bronze layer in Spark, cleaned and joined into a Silver layer, aggregated into a Gold layer, and finally published to a Curated layer that exposes business-ready tables consumed by the Spark MLlib price-prediction model and a summary analytics view.

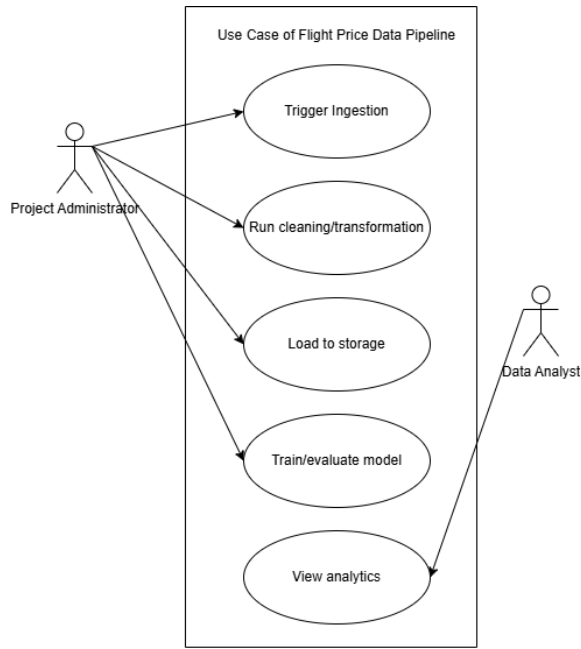


Fig. 1: Use Case Diagram of Flight Price Data Pipeline.

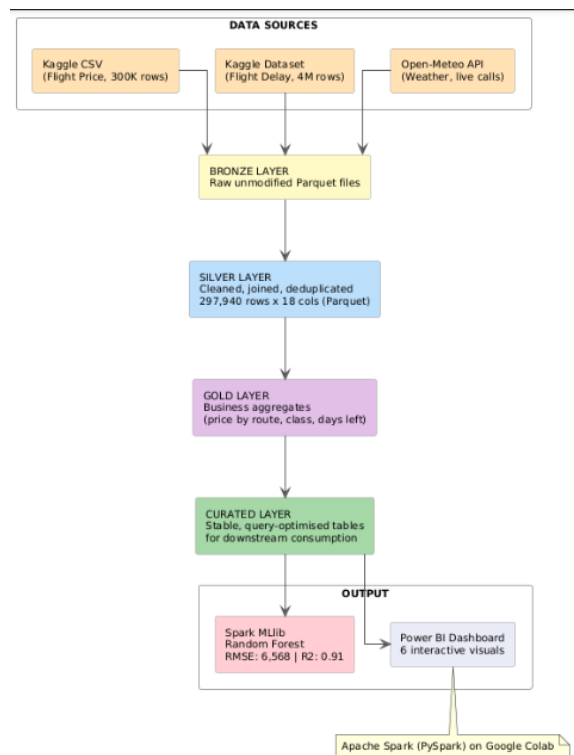


Fig. 2: System Architecture Diagram of Flight Price Data Pipeline.

C. Database Design

Two complementary data models will be used. For the source layer, an entity-relationship diagram (ERD), shown in Fig. 3, represents the raw tables linked by City and Date keys. For the analytical layer, a simple star-schema-style multidimensional model, shown in Fig. 4, is designed with a central Flight Booking fact table connected to dimension tables for Airline, Route, Date, and Weather Condition, which will allow Spark SQL queries to aggregate price by any combination of these dimensions.

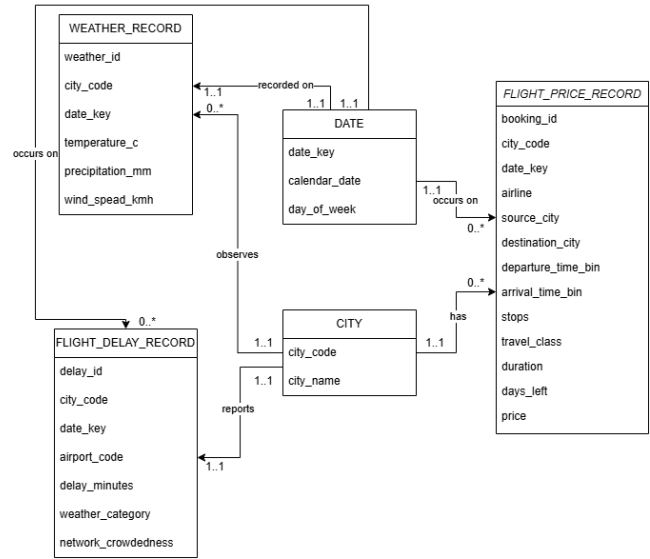


Fig. 3: Entity-Relationship Diagram of Flight Price Data Pipeline.

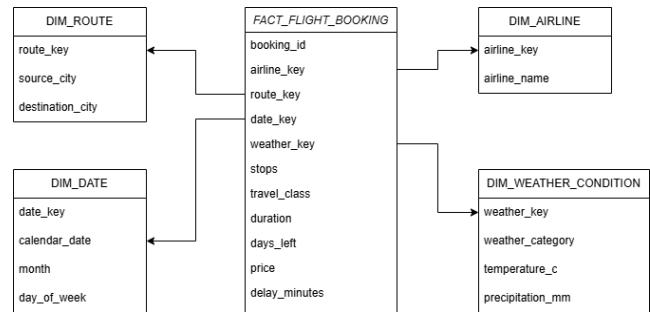


Fig. 4: Star Schema Diagram of Flight Price Data Pipeline.

D. Main Interface and Testing

The main interface for this individual project is a notebook-style Spark application (PySpark) supplemented by a lightweight summary view of average price by route, airline and month, generated after every pipeline run. Testing is carried out at two levels, which are unit-level checks on the transformation functions and pipeline-level checks that re-run the full ETL on a small sample and compare the row count and schema of the output against expected values.

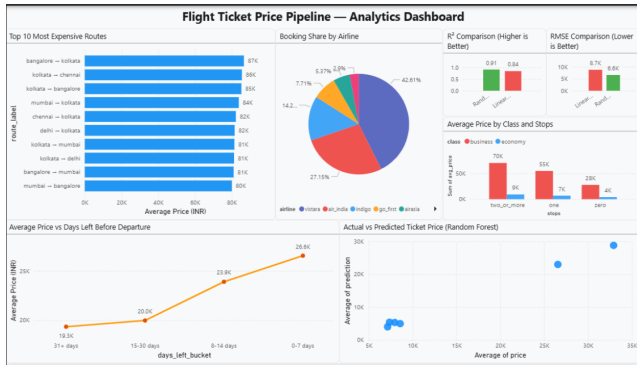


Fig. 5: Main Interface Mock-up of Flight Price Data Pipeline.

V. IMPLEMENTATION

The design described in Section IV was implemented and executed on Google Colab using PySpark. The three sources were ingested into the Bronze layer as raw Parquet files, after which the Silver-layer cleaning and join reduced the combined dataset to 297,940 usable records across 18 unified columns, matching the schema planned in Section III-D. The Gold layer next produced three business-aggregated summary tables, and the Curated layer republished these as the stable, query-optimized tables consumed downstream as what is shown Figure 6.

```
Price rows after cleaning: 297940
Industry avg dep delay (Feb-Mar): 14.89
Industry avg arr delay (Feb-Mar): 14.79
Industry cancellation rate: 0.0293
Silver rows: 297940
Silver columns: ['source_city', 'destination_city', 'airline', 'departure_time', 'stops', 'arrival_time', 'class', 'duration',
                'source_city|destination_city|airline|departure_time|stops|arrival_time|class|duration|days_left|price|is_price_outlier']
delhi|    |mumbai|air_india|    evening|one|    evening|economy|    22.75|    1|19815.0|    false
delhi|    |mumbai|airasia|    evening|one|early_morning|economy|    12.25|    2| 9949.0|    false
delhi|    |mumbai|air_india|early_morning|one|    morning|economy|    26.42|    5|12259.0|    false
only showing top 3 rows
Silver layer saved.
```

Fig. 6: Silver-layer output showing cleaned, joined dataset (297,940 rows, 18 columns) with delay and weather features merged.

A Random Forest regressor was trained using Spark MLlib on the Curated-layer features, together with a linear regression model trained on the same split as the baseline for comparison. The train-test split was performed using the days-left-before-departure field as a proxy, so that the model was evaluated only on booking patterns it had not seen during training. The resulting summary tables and model outputs were exported to CSV and connected to Power BI to produce the six-visual analytics dashboard shown in Fig. 5, covering top routes by price, booking share by airline, and more, as what is shown in Figure 7.

```
Curated rows: 297940
Curated columns: ['source_city', 'destination_city', 'airline', 'departure_time', 'stops', 'arrival_time', 'class', 'duration',
                  'source_city|destination_city|airline|departure_time|stops|arrival_time|class|duration|days_left|price|is_price_outlier']
delhi|    |mumbai|air_india|    evening|one|    evening|economy|    22.75|    1|19815.0|    false
delhi|    |mumbai|airasia|    evening|one|early_morning|economy|    12.25|    2| 9949.0|    false
delhi|    |mumbai|air_india|early_morning|one|    morning|economy|    26.42|    5|12259.0|    false
only showing top 3 rows
Curated layer saved.
```

Fig. 7: Gold-layer aggregation output.

VI. CONCLUSION

This project set out to do three things: building a Spark-based ETL pipeline that pulls together three different data sources, storing the processed data in a structured medallion architecture, and layering a machine learning model on top to predict flight ticket prices. Looking back at where things started and where they ended up, all three objectives were met. The pipeline successfully ingested 300,153 flight booking records from the US Bureau of Transportation Statistics dataset, and daily weather data for six Indian departure cities fetched directly from the Open-Meteo API, which are three different source types as required. After cleaning and joining them in the Silver layer, the dataset came down to 297,940 usable records with 18 unified columns covering pricing, operational delay context and weather conditions. The Gold layer produced three business-ready summary tables, and the Curated layer published a stable, model-ready version enriched with route-level average prices. The entire pipeline ran within Apache Spark on Google Colab, which confirms that a distributed data engineering workflow of this kind is accessible without specific setup or infrastructure.

The analytics from the Gold layer alone told an interesting story. Flights booked within seven days of departure cost an average of 37 percent more than those booked over a month in advance. This is technically a clear and practically useful finding for any traveler or pricing analyst. Vistara consistently dominated the most expensive routes across all city pairs, while budget carriers like AirAsia and IndiGo clustered at the lower end. Business class with one stop was the most expensive booking segment at an average of over 54,000 INR, nearly double the economy equivalent.

On the prediction side, the Random Forest model achieved an R^2 of 0.91 and an RMSE of 6,568.75, meaning it could explain 91 percent of the variance in ticket prices on unseen test data. This outperformed the linear regression baseline, which achieved an R^2 of 0.84 and an RMSE of 8,742.76, with a difference of roughly 25 percent in error. The train-test split was done chronologically using days left before departure as a proxy, so the model was genuinely evaluated on booking patterns it had not seen during training. These results are consistent with what Wong et al. [6] found on a much larger dataset, which gives some confidence that the approach generalizes beyond this specific data sample.

That said, there are a few honest limitations worth acknowledging. The delay dataset came from US domestic flights, not Indian routes, so the delay benchmarks joined into the pipeline are industry-level averages rather than route-specific figures. This was a practical workaround due to the unavailability of a publicly accessible Indian flight delay dataset with the required granularity, and it is the most significant gap between what the pipeline was designed for and what is achieved. Moreover, the Kaggle pricing dataset only covers a six-week window in early 2022, which means the model has not seen seasonal variation such as prices during monsoon season or major Indian holidays which could behave differently.

For future work, three improvements can be made. First, sourcing an Indian-specific delay dataset would make the operational features far more insightful. Second, testing Gradient-Boosted Trees alongside Random Forest with proper cross-validation would give a more accurate comparison of model performance. Third, extending the pipeline with Spark Structured Streaming would allow new booking records to be processed in near real-time rather than as a batch, which brings the system closer to how production-grade pricing pipelines operate in industry.

Overall, this project demonstrated that a functional, multi-source flight price data pipeline can be built entirely within Apache Spark, following a Bronze-Silver-Gold-Curated medallion architecture, and that the resulting dataset is rich enough to support both meaningful analytical insights and a reasonably accurate price prediction model.

REFERENCES

- [1] S. Bathwal, "Flight price prediction" [Dataset], Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction>
- [2] A. Bubalo and A. A. Gaggero, "Flight delays in European airline networks," *Research in Transportation Business & Management*, vol. 41, p. 100631, 2021, doi: 10.1016/j.rtbm.2021.100631.
- [3] J. Calzada and X. Fageda, "Airport dominance, route network design and flight delays," *Transportation Research Part E: Logistics and Transportation Review*, vol. 170, p. 103000, 2023, doi: 10.1016/j.tre.2022.103000.
- [4] N. R. Mandala, "The evolution of ETL architecture: From traditional data warehousing to real-time data integration," *World Journal of Advanced Research and Reviews*, vol. 1, no. 3, pp. 73–84, 2019, doi: 10.30574/wjarr.2019.1.3.0033.
- [5] R. Mulla, "Flight delay dataset 2018–2022" [Dataset], Kaggle, 2022. [Online]. Available: <https://www.kaggle.com/datasets/robikscube/flight-delay-dataset-20182022>
- [6] P. Wong, P. Thant, P. Yadav, R. Antaliya, and J. Woo, "Using Spark machine learning models to perform predictive analysis on flight ticket pricing data," arXiv preprint, 2023, doi: 10.48550/arXiv.2310.07787.
- [7] P. Zippenfenig, "Open-Meteo.com weather API" [Software], Zenodo, 2023, doi: 10.5281/zenodo.7970649.

APPENDIX

TASKS	DURATION (WEEKS)	25/5-29/5	1/6-5/6	8/6-12/6	15/6-19/6	22/6-26/6	29/6-3/7	6/7-10/7	13/7-17/7
Topic confirmation and dataset selection	1								
Literature review and methodology design	2								
System design planning	2								
Draft report writing	3								
Draft report submission	1								
Proposal presentation	1								
Sprint 1 — data ingestion, Bronze layer	2								
Sprint 2 — cleaning and transformation, Silver layer	1								
Sprint 3 — Gold and Curated layers	2								
Sprint 4 — AI model integration	1								
Final report writing	1								
Final report submission	1								
Demo and presentation	1								

Fig. 8: Project Gantt Chart / Task Timeline.